Client-Side Scripting II

Presented by: Carlo Mamo

Bachelors of Science (Hons) (MQF Level 6)

Chapter 5 – Animations and Transitions

Agenda

- Transitions and Animations
- VUE’s transition component
- Simple transition Example
- Page transitions
- Group transitions

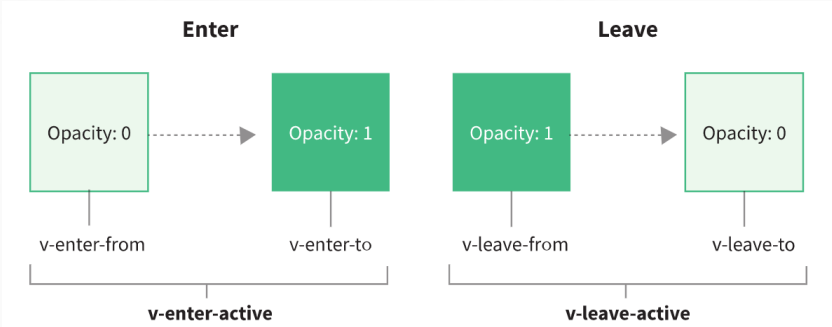
VUE Transitions and Animations

- VUE Transitions and Animations are a great way to make your website **feel more modern** and to give site visitors **a better user experience**. A well-designed transition can:
  - Capture and direct your user's attention
  - Emphasize important information e.g. clicking a particular button
  - Suggest a natural flow on your website
  - Guide users around your page
  - Help create a more professional brand image

What are transitions and animations

- A **transition** is simply the change that occurs while a thing **moves between two states**. The first state is the starting point, and the second state is the ending point. E.g. from visible to invisible.

- Transitions require **6 CSS classes** to be added and removed **at the right time** for that transition to work. e.g. in the below we are transitioning (entering) from invisible to visible on to the screen and then transitioning (leaving) away from the screen.



VUE's transition component

- Often we are transitioning to and away from the default style of a particular HTML element. The **default style** of an HTML element is already **opacity: 1** because **its visible by default**.
- Same thing applies for scale. The default **scale** of an HTML element is **100%**.
- Hence we **do not need to set opacity: 1 or scale: 100% in v-enter-to or v-leave-from** on our elements because that is already taken care of by the browser.

Entering transition

- When designing transitions the starting style should be: **opacity: 0**

```
.v-enter-from { /* starting style */
    opacity: 0;
}
```

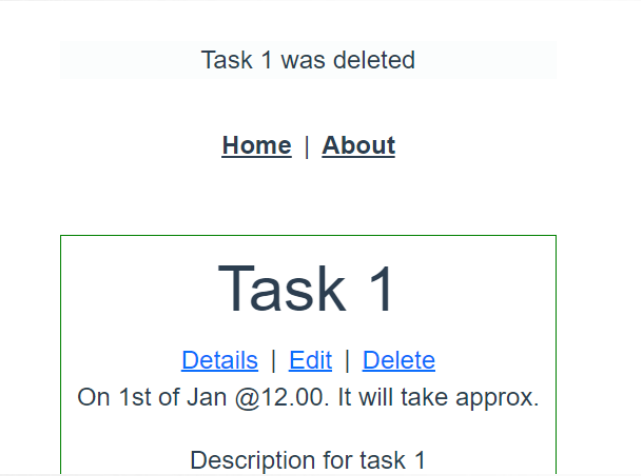
- Then we use the **v-enter-active** to define the behavior of the transition mainly the **duration and easing** while it is happening (active).

```
.v-enter-active { /* active entering style */
    transition: opacity 2s ease-in;
}
```

- Duration in the above is being set to last 2 seconds with an ease-in meaning it will fade in slowly and then get faster as it approaches 1 - https://www.w3schools.com/cssref/css3_pr_transition-timing-function.asp)

Example

- After clicking the delete button we now want to display the message and apply a fade transition to provide more context of where the element came from and what is happening.




```
<transition name="slide-fade">
  <div id="flashMessage" v-if="GStore.flashMessage">
    {{ GStore.flashMessage }}
  </div>
</transition>
```

Example

- Each of the css classes will be prefixed with the name of the transition. The **v-** prefix is the default when you use a **<transition>** element with no name. If you use **<transition name="fade">** for example, the **v-enter-from** class would instead be **fade-enter-from.**
- It's good practice to name the transitions according to what they do e.g. **fade.**

```
/* FADE TRANSITION */
/* What style are we starting from and leaving to? Invisible*/
/* We are transitioning to opacity 1 which is the default so we do not need to add this*/
.fade-enter-from,
.fade-leave-to {
  opacity: 0;
}
/* What is the active entering and leaving style? */
.fade-enter-active {
  transition: opacity 1s ease-in;
}
.fade-leave-active {
  transition: opacity 1s ease-in;
}
```

Exercise

- Instead of the fade transition try to apply another two different transitions:
  - **Slide-fade** (message slides in from the left and disappears on the right)
  - **Slide-up** (message slides in from bottom to up and disappears sliding up)

Page transitions

- Currently when the user navigates between routes e.g. from Tasks to the About page, the route just pops up. We can smooth this up with **a page transition** using a fade animation.
- Since the transition css code is in the App.vue this can be used throughout the entire application. Hence this is a **global** transition.
- The **<router-view>** component which is replaced by the component that we navigate to, can be wrapped by the transition component so that the transition is applied whenever the page is changed.

```
<router-view v-slot="{ Component }">
  <transition name="slide-fade" mode="out-in">
    <component :is="Component" />
  </transition>
</router-view>
```

Page transitions – slide-fade

- To achieve a slide fade transition **transform: translateX(10px)** should be used. **v-leave-to** and **v-enter-from** should be separate in the style section since value for transform attribute is different. **Opacity** should be changed to **all** since it should affect both transform and opacity.

```
/* SLIDE TRANSITION */

.slide-fade-leave-to {
  transform: translateX(10px);
  opacity: 0;
}

.slide-fade-enter-from{
  transform: translateX(-10px);
  opacity: 0;
}

.slide-fade-enter-active,
.slide-fade-leave-active {
  /* transition: opacity 1s ease-out; */
  transition: all 1s ease;
}
```

CSS: Transition timing functions

- CSS3's transitions and animations support easing, formally called a **timing function**.

`ease-in` will start the animation slowly, and finish at full speed.

`ease-out` will start the animation at full speed, then finish slowly.

`ease-in-out` will start slowly, be fastest at the middle of the animation, then finish slowly.

`ease` is like `ease-in-out`, except it starts slightly faster than it ends.

`linear` uses no easing.

Group transitions

- The same transition can be applied to a list of elements or components. Note the **appear** keyword which specifies whether to apply the same transition when the page is first rendered.

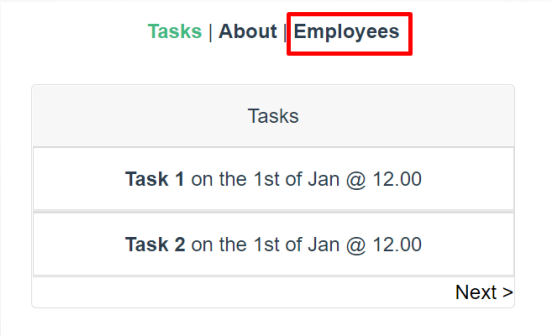
```
<div>
  <input type="text" v-model="newEmployee" placeholder="Name" />
  <button @click="addEmployee">Add Employee</button>
  <transition-group tag="ul" name="slide-up" appear class="list-group list-group-flush">
        <li class="list-group-item" v-for="emp in employees" :key="emp">{{ emp }}</li>
  </transition-group>
</div>
```

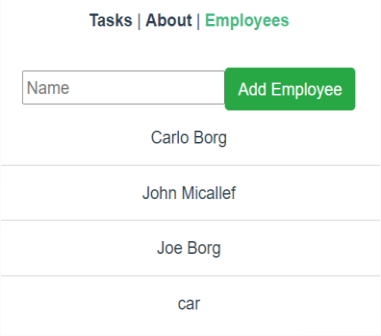
```
/* SLIDE-UP TRANSITION */

.slide-up-enter-from{
  transform: translateY(10px);
  opacity: 0;
}

.slide-up-enter-active {
  /* transition: opacity 1s ease-out; */
  transition: all 1s ease;
}
```

Exercise

- Add another route (page) to the menu item. Create a new component (Employee.vue) which must have a list of employees. Create an input field which must be available for the user to be able to add a new employee to the list.

- Apply a **slide-up transition** so that when the user adds a new name to the list, the name is displayed on screen sliding up.





References

Vuejs.org. 2021. *Vue.js*. [online] Available at: <https://vuejs.org/> [Accessed 7 February 2021].

Schweiger, P., 2021. *Vue Mastery*. [online] Vue Mastery. Available at: <https://www.vuemastery.com/> [Accessed 7 February 2021].

Vue School. 2021. *Learn Vue.js from core-team members and industry experts at Vue School*. [online] Available at: <https://vueschool.io/> [Accessed 7 February 2021].

Udemy. 2021. *Develop with VueJS 2 (Complete Vue.js Router and Vuex Course)*. [online] Available at: <https://www.udemy.com/course/vuejs-2-the-complete-guide/> [Accessed 7 February 2021].

Any questions?

THANK YOU

Email: carlo.mamo@mcast.edu.mt